



武汉大学
Wuhan University

第二章 数字技术基础

2.1 数制与编码

武汉大学计算机学院 2025-2026第一学期



主要内容

Main Content

进位计数制

数制转换

带符号二进制数的代码表示

几种常用的编码

2.1.1 进位计数制

一、概念

采用若干位**数码**进行计数，并规定**进位规则**的科学计数法称为**进位计数制**，简称**数制**。

数码的个数
和计数规律
是进位计数
制的两个决
定因素

数制的
要素

数码：表示基本**数值大小**的不同**数字符号**

进位规则：逢几进一，
借一当几

基数：数码的个数

位权：每个位置上单
位数码代表的数值

一、概念

十进制数：

数码： 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

基数： 10（数码的个数）

进位规则： 逢十进一，借一当十

位权： 每个位置代表的数值大小

..., 10^4 , 10^3 , 10^2 , 10, 1, 10^{-1} , 10^{-2} ,

...

任意十进制数 $a_{n-1}a_{n-2}\cdots a_0.a_{-1}a_{-2}\cdots a_{-m}$

的按位权展开式：

$$\begin{aligned}(N)_{10} &= a_{n-1} \times 10^{n-1} + a_{n-2} \times 10^{n-2} + \cdots + a_0 \times 10^0 \\ &\quad + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \cdots + a_{-m} \times 10^{-m} \\ &= \sum_{i=-m}^{n-1} a_i \cdot 10^i\end{aligned}$$

其中：

n 为整数位数, m 为小数位数

a_i 为 0, 1, ..., 9

■ 实例：

$$(563.48)_{10} = 5 \times 10^2 + 6 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 8 \times 10^{-2}$$

二、数字系统中的常用数制

➤ 二进制 $(1101.01)_2$

➤ 八进制 $(15.2)_8$

➤ 十六进制 $(D.4)_{16}$

◆ 数字系统内的数是以器件的**物理状态**来表示的, 二进制数只需要**两种不同状态**即可表示, 因此数字系统采用二进制数

◆ 书写时, 二进制数往往很长; 为书写方便, 常采用八进制或十六进制数

2.1.1 进位计数制

二、数字系统中的常用数制——二进制

	十进制	二进制
数 码	0,1,2,3,4,5,6,7,8,9	0, 1
基 数	10	2
实 例	13.25	1101.01
位 权	10^i	2^i
运算规则	逢十进一 借一当十	逢二进一 借一当二
按 权 展 开	$(N)_{10} = \sum_{i=-m}^{n-1} a_i \cdot 10^i$	$(N)_2 = \sum_{i=-m}^{n-1} b_i \cdot 2^i$

二、数字系统中的常用数制——二进制

任意二进制数 $b_{n-1}b_{n-2}\cdots b_0.b_{-1}b_{-2}\cdots b_{-m}$

的按位权展开式:

$$\begin{aligned}(N)_2 &= b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \cdots + b_0 \times 2^0 \\ &\quad + b_{-1} \times 2^{-1} + b_{-2} \times 2^{-2} + \cdots + b_{-m} \times 2^{-m} \\ &= \sum_{i=-m}^{n-1} b_i \cdot 2^i\end{aligned}$$

其中:

n 为整数的位数, m 为小数位数, b_i 为0或1.

■ 例如:

$$(1101.01)_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$$

2.1.1 进位计数制

二、数字系统中的常用数制——八进制

数码	0,1,2,3,4,5,6,7
基数	8
实例	76.02
位权	8^i
运算规则	逢八进一 借一当八
按权展开	$(N)_8 = \sum_{i=-m}^{n-1} b_i \cdot 8^i$

2.1.1 进位计数制

二、数字系统中的常用数制——十六进制

数码	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
基数	16
实例	1A6.D2
位权	16^i
运算规则	逢十六进一 借一当十六
按权展开	$(N)_{16} = \sum_{i=-m}^{n-1} b_i \cdot 16^i$

2.1.2 数制转换

一、R进制转换成十进制

- 数码: 基本符号 $0, 1, 2, \dots, (R-1)$
- 基数: R
- 位权: R^i
- 运算规则: 逢 R 进一, 借一当 R .

将R进制数按位权展开, 再按十进制进行计算, 即得到对应的十进制数。

$$\begin{aligned}\text{例如: } (53.62)_7 &= 5 \times 7^1 + 3 \times 7^0 + 6 \times 7^{-1} + 2 \times 7^{-2} \\ &= (38.898)_{10}\end{aligned}$$

$$\begin{aligned}(24.3)_5 &= 2 \times 5^1 + 4 \times 5^0 + 3 \times 5^{-1} \\ &= (14.6)_{10}\end{aligned}$$

2.1.2 数制转换

二、十进制转换成R进制

整数部分：

基数连除法 (除以基数 R 取余数)

除基取余

商零为止

例：

$$(25)_{10} = (\quad)_2$$

2	25	余 1	低位 ↑ 高位
2	12	余 0	
2	6	余 0	
2	3	余 1	
2	1	余 1	
	0		

$$\therefore (25)_{10} = (11001)_2$$

2.1.2 数制转换

二、十进制转换成R进制

整数部分：

基数连除法 (除以基数
数R取余数)

除基取余
商零为止

$$\begin{array}{r} 16 \overline{) 54} \dots \dots \text{余 } 6 \\ 16 \overline{) 3} \dots \dots \text{余 } 3 \\ 0 \end{array}$$

低位
↑
高位

例：

$$\therefore (54)_{10} = (36)_{16}$$

$$(54)_{10} = (\quad)_{16}$$

2.1.2 数制转换

二、十进制转换成R进制

小数部分：

基数连乘法 (乘以基数 R 取整数)

乘基取整，直至满足精度要求或乘积小数部分为0

例：

$$(0.125)_{10} = (\quad)_2$$

高位	0.125	
	$\times \quad 2$	
	0.25	
	$\times \quad 2$	
	0.5	
	$\times \quad 2$	
	1.0	
↓		
低位		

$$\therefore (0.125)_{10} = (0.001)_2$$

2.1.2 数制转换

二、十进制转换成R进制

小数部分：

基数连乘法 (乘以基数 R 取整数)

乘基取整，直至满足精度要求或乘积小数部分为0

例：

$$(0.93)_{10} = (\quad)_2$$

高位

低位

$$\begin{array}{r}
 0.93 \\
 \times 2 \\
 \hline
 1.86 \\
 \times 2 \\
 \hline
 1.72 \\
 \times 2 \\
 \hline
 1.44 \\
 \times 2 \\
 \hline
 0.88 \\
 \times 2 \\
 \hline
 1.76 \\
 \dots
 \end{array}$$

$$\therefore (0.93)_{10} = (0.11101)_2$$

二、十进制转换成R进制——总结

整数部分转换：除以基数 R 取余数

- a. 整数部分除以基数 R ，余数做为等值的 R 进制数最低位；
- b. 将商再除以 R ，余数做为等值的 R 进制数的次低位；
- c. 重复步骤b，直到商等于零为止。

小数部分转换：乘以基数 R 取整数

- a. 小数部分乘以基数 R ，积的整数部分为 R 进制数的最高位；
- b. 将小数部分再乘以基数 R ，其积的整数部分为次高位；
- c. 重复步骤b，直到达到要求的精度为止。

2.1.2 数制转换

三、二进制与八进制、十六进制之间的转换



◆ 每四位二进制数对应一位十六进制数

◆ 每三位二进制数对应一位八进制数

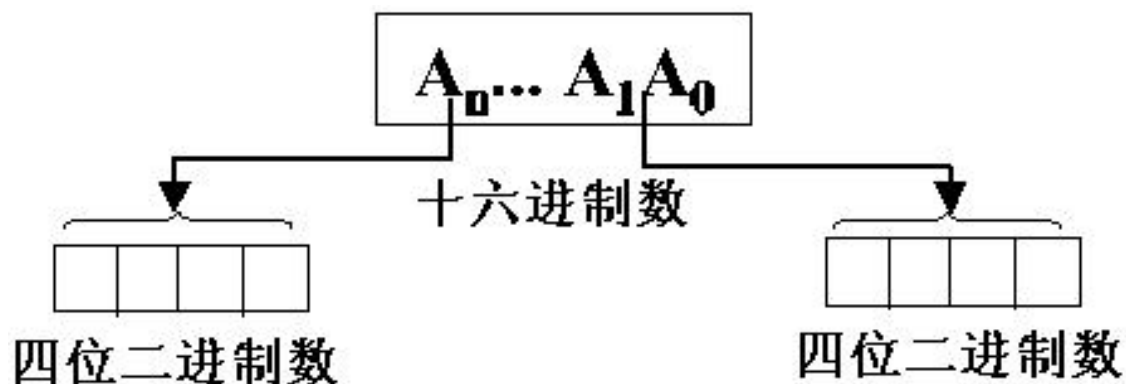
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

2.1.2 数制转换

三、二进制与八进制、十六进制之间的转换

十六进制 $\longrightarrow$ 二进制

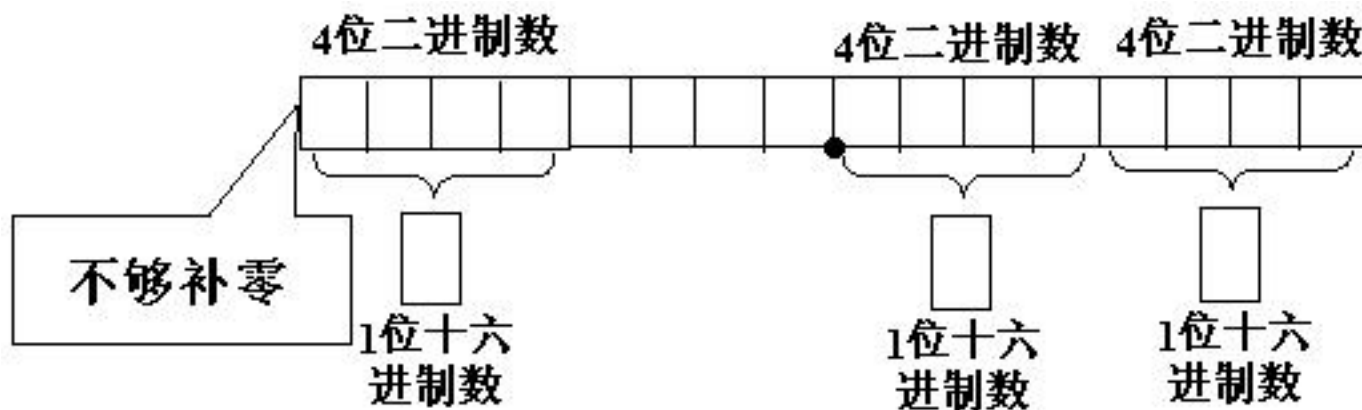


例： $(7D.A6)_{16} = (0111\ 1101 . 1010\ 0110)_2$
 $= (1111101.1010011)_2$

2.1.2 数制转换

三、二进制与八进制、十六进制之间的转换

二进制 $\longrightarrow$ 十六进制



$$\begin{aligned} \text{例: } (1011011.01)_2 &= (\underline{0101} \ \underline{1011}.\underline{01}\underline{00})_2 \\ &= (5B.4)_{16} \end{aligned}$$

三、二进制与八进制、十六进制之间的转换

■ 二进制数转换为八进制数

从小数点起每三位分一组，不足三位补零

$$\text{例: } (11010101.01011)_2 = (011\ 010\ 101 . 010\ 110)_2 = (325.26)_8$$

$$(111001010.100101)_2 = (111\ 001\ 010 . 100\ 101)_2 = (712.45)_8$$

■ 八进制数转换为二进制数

将每一位数表示成三位二进制数

$$\text{例: } (325.26)_8 = (011\ 010\ 101 . 010\ 110)_2 = (11010101.01011)_2$$

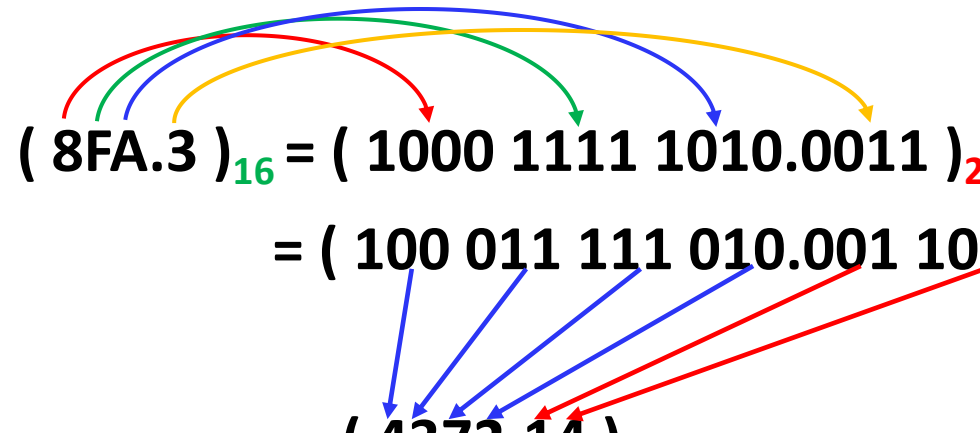
$$(712.45)_8 = (111\ 001\ 010 . 100\ 101)_2$$

2.1.2 数制转换

三、二进制与八进制、十六进制之间的转换

十六进制 $\longleftrightarrow$ 二进制 $\longleftrightarrow$ 八进制

例1: $(8FA.3)_{16} = (1000\ 1111\ 1010.0011)_2$
 $= (100\ 011\ 111\ 010.001\ 100)_2$
 $= (4372.14)_8$



例2: $(712.43)_8 = (111\ 001\ 010.100\ 011)_2$
 $= (0001\ 1100\ 1010.1000\ 1100)_2$
 $= (1CA.8C)_{16}$

主要内容

Main Content

概述

数制及其转换

带符号二进制数的代码表示

几种常用的编码

2.1.3 带符号二进制数的代码表示

- 无符号二进制数

- 特点：无符号位

$$A_{n-1}A_{n-2}\dots A_1A_0$$

表示范围： $0\sim 2^n-1$

- 有符号二进制数

- 原码
- 反码
- 补码

符号位

数值部分

有符号数的最高位表示符号，
0表示正数，1表示负数

2.1.3 带符号二进制数的代码表示

- **原码**：最高位为符号位，**0**表示正数，**1**表示负数
数值位就是自然二进制码

例：正数 $[+18]_{\text{原}} = \mathbf{0} \ 10010$

负数 $[-18]_{\text{原}} = \mathbf{1} \ 10010$

- **反码**：正数的反码与原码相同；
负数的反码为**原码按位取反** (数值部分)

例：正数 $[+18]_{\text{反}} = \mathbf{0} \ 10010$

负数 $[-18]_{\text{反}} = \mathbf{1} \ 01101$

- **补码**：正数的补码与原码相同；
负数的补码为**反码+1** (数值部分)

例：正数 $[+18]_{\text{补}} = \mathbf{0} \ 10010$

负数 $[-18]_{\text{补}} = \mathbf{1} \ 01110$

二进制加法器与逻辑门电路的“天然适配”

- 计算机的物理核心是数字电路
- 在所有运算中，加法是唯一能直接通过简单电路实现的基础运算，其他运算无法直接用基础电路完成
- 二进制加法的极简性
 - $0 + 0 = 0$ （无进位）
 - $0 + 1 = 1$ （无进位）
 - $1 + 0 = 1$ （无进位）
 - $1 + 1 = 10$ （有进位，结果为 0，进位 1）
 - 这种简单规则可以通过最基础的逻辑门电路（与门、或门、异或门）组合实现
- 减法、乘法、除法等运算的规则远比加法复杂，无法通过简单逻辑门直接实现
- 硬件设计的最优解是：将所有复杂运算拆解为加法，复用加法器这一基础硬件，为每种运算单独设计复杂电路（会导致硬件体积、功耗、成本剧增）

- 1. 减法 → 加法：补码的核心作用
 - 计算机中没有专门的“减法器”，减法通过“加负数的补码”实现。
 - 对于 n 位二进制数，某个数 x 的“补码” = $2^n - x$ （模 2^n 运算）
 - “ $a - b$ ”可转化为“ $a + (2^n - b)$ ”（即 a 加 b 的补码），最终结果仍符合模 2^n 的规则
 - 例如，4 位二进制中，计算 $5 - 3$ （即 $5 + (-3)$ ）：
 - 5 的二进制：0101
 - 3 的补码（4 位）：1101（ $2^4 - 3 = 13$ ，二进制 1101）
 - 相加：0101 + 1101 = 10010（舍弃超出 4 位的进位 1，结果为 0010，即十进制 2，与 $5 - 3 = 2$ 一致）
 - 补码的引入，让减法彻底融入加法体系，无需额外硬件

- 2. 乘法 → 加法：移位 + 累加

- 乘法的本质是“相同数的多次相加”，而二进制的移位操作（左移 1 位 $= \times 2$ ，左移 2 位 $= \times 4$ ）可大幅简化累加次数。
- 例如，计算 $a \times b$ （二进制）：
- 将 b 拆分为“2 的幂次之和”（如 $b=5$ ，二进制 $101=4+1=2^2+2^0$ ）；
- 则 $a \times b = a \times (2^2 + 2^0) = (a \ll 2) + (a \ll 0)$ （左移 2 位后加左移 0 位）；
- 最终通过多次加法完成计算
- 以 8×5 （二进制 1000×101 ）为例：
- $101 = 100 + 01 \rightarrow$ 对应左移 2 位和左移 0 位；
- 计算： $(1000 \ll 2) + (1000 \ll 0) = 100000 + 1000 = 101000$
（十进制 40，与 $8 \times 5 = 40$ 一致）
- 计算机的乘法器本质就是“移位器 + 加法器”的组合，通过循环移位和累加完成乘法

- 3. 除法 $\rightarrow$ 加法：减法（补码加法）的循环
 - 除法的本质是“多次减法”（求商是“能减多少次”，求余数是“最后剩多少”），而减法已转化为加法（补码）。
 - 例如，计算 $a \div b$ （求商 q 和余数 r ）：
 - 循环用 a 减去 b ，每减一次商 q 加 1；
 - 直到 $a < b$ 时停止，此时 a 即为余数 r ；
 - 二进制除法中，通过“比较 + 移位减法”优化效率（如“恢复余数法”“不恢复余数法”），但核心仍是“多次补码加法”
 - 例如 $10 \div 3$ ：
 - $10 - 3 = 7$ ($q=1$) $\rightarrow 7 - 3 = 4$ ($q=2$) $\rightarrow 4 - 3 = 1$ ($q=3$) $\rightarrow 1 < 3$ 停止；
 - 结果：商 $q=3$ ，余数 $r=1$ ，与 $10 \div 3 = 3$ 余 1 一致

主要内容

Main Content

概述

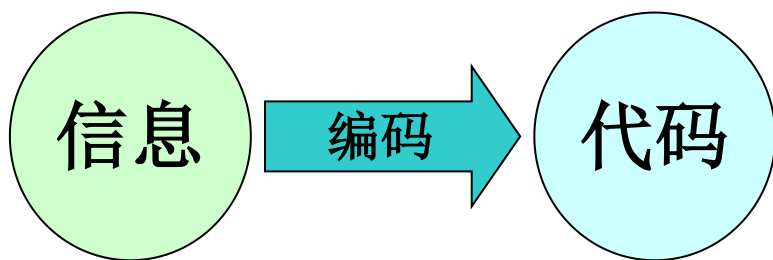
数制及其转换

带符号二进制数的代码表示

几种常用的编码

2.1.4 几种常用的编码

编码：用若干位**二进制数码**表示数、字母和符号等信息。



n 位二进制代码共有 2^n 种不同的组合，能够对 2^n 种不同信息进行编码

◆ 二进制编码

自然二进制码、格雷码

◆ 二—十进制编码

8421码、余3码、BCD循环码

◆ 可靠性编码

奇偶校验码

2.1.4 几种常用的编码

一、十进制数的二进制编码（BCD码）

- 用4位二进制代码对十进制数字符号进行编码，简称为**二 - 十进制代码**，或称BCD(Binary Coded Decimal)码。
- 根据代码中每一位是否有固定的权，通常将BCD码分为**有权码**和**无权码**两种类型。
- BCD码既有二进制的形式，又有十进制的特点。常用的BCD码有**8421码**、**2421码**和**余3码**。

2.1.4 几种常用的编码

一、十进制数的二进制编码（BCD码）

常用的3种BCD码

十进制字符	8421码	2421码	余3码
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

2.1.4 几种常用的编码

一、十进制数的二进制编码（BCD码）

1. 8421码

用4位二进制码表示一位十进制字符的一种**有权码**，4位二进制码从高位至低位的权依次为 2^3 、 2^2 、 2^1 、 2^0 ，**即为8、4、2、1，故称为8421码。**

按8421码编码的0~9与用4位二进制数表示的0~9完全一样。所以，8421码是一种人机联系时广泛使用的中间形式。

注意：

8421 BCD码中不允许出现1010~1111六种组合（因为没有十进制数字符号与其对应）。

2.1.4 几种常用的编码

一、十进制数的二进制编码（BCD码）

1. 8421BCD码

(1) 8421BCD码与十进制数之间的转换

8421BCD码与十进制数之间的转换是**按位**进行的，即十进制数的每一位与4位二进制编码对应。例如：

$$(258)_{10} = (0010 \ 0101 \ 1000)_{8421\text{码}}$$

$$(0001 \ 0010 \ 0000 \ 1000)_{8421\text{码}} = (1208)_{10}$$

(2) 8421BCD码与二进制的区别

例如：

$$(28)_{10} = (11100)_2 = (\underline{0010} \ \underline{1000})_{8421}$$

2.1.4 几种常用的编码

一、十进制数的二进制编码（BCD码）

2. 2421码

是用4位二进制码表示一位十进制字符的一种有权码，4位二进制码从高位至低位的权依次为2、4、2、1，故称为2421码。

若一个十进制字符X的2421码为 $a_3 a_2 a_1 a_0$ ，则该字符的值为：

$$X = 2a_3 + 4a_2 + 2a_1 + 1a_0$$

$$\text{例如, } (1101)_{2421\text{码}} = (7)_{10}$$

2421码与十进制数之间的转换同样是按位进行的，例如：

$$(258)_{10} = (0010 \ 1011 \ 1110)_{2421\text{码}}$$

$$(0010 \ 0001 \ 1110 \ 1011)_{2421\text{码}} = (2185)_{10}$$

2.1.4 几种常用的编码

一、十进制数的二进制编码（BCD码）

2. 2421码

(1) 2421码不具备单值性。例如，0101和1011都对应十进制数字5。为了与十进制字符一一对应，2421码不允许出现0101~1010的6种状态。

(2) 2421码是一种对9的自补代码。即一个数的2421码只要自身按位变反，便可得到该数对9的补数的2421码。例：

$$\begin{array}{ccc} (4)_{10} & \longrightarrow & (0100)_{2421} \\ & \Downarrow & \\ & (1011)_{2421} & \longrightarrow (5)_{10} \end{array}$$

具有这一特征的BCD码可给运算带来方便，因为直接对BCD码进行运算时，可利用其对9的补数将减法运算转化为加法运算。

2.1.4 几种常用的编码

二、可靠性编码

作用：减少或者发现代码在形成和传送过程中可能发生的错误，提高系统的可靠性。

1. 格雷(Gray)码

特点：任意两个相邻的数，其格雷码仅有一位不同。

4位二进制码对应的典型格雷码

十进制数	4位二进制码	典型格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

2.1.4 几种常用的编码

二、可靠性编码

2. 奇偶校验码

是一种用来检验代码在传送过程中是否产生错误的代码。

(1) 组成:

信息位——位数不限的一组二进制代码

奇偶检验位——仅有一位

(2) 编码方式: 有两种编码方式

奇检验: 使信息位和检验位中 “1”的个数 共计为奇数;

偶检验: 使信息位和检验位中 “1”的个数 共计为偶数。

例如:

信息位 (7位)	采用奇检验的检验位 (1位)	采用偶检验的检验位 (1位)
1001100	0	1

2.1.4 几种常用的编码

二、可靠性编码

8421码的奇偶检验码

十进制 数码	采用奇检验的8421码		采用偶检验的8421码	
	信息位	检验位	信息位	检验位
0	0000	1	0000	0
1	0001	0	0001	1
2	0010	0	0010	1
3	0011	1	0011	0
4	0100	0	0100	1
5	0101	1	0101	0
6	0110	1	0110	0
7	0111	0	0111	1
8	1000	0	1000	1
9	1001	1	1001	0

- ◆ 码字中出现奇数个错误时，能检测出来
- ◆ 不能确定出错码位个数
- ◆ 不能确定错误的具体位置

2.1.4 几种常用的编码

三、字符编码

ASCII码（美国信息交换标准码）

用7位二进制数编码
128个字符，包括大、小写字母，数字0到9，标点符号，以及在美国式英语中使用的特殊控制字符等。

7 位ASCII码编码表

低4位代码 ($a_4a_3a_2a_1$)	高3位代码 ($a_7a_6a_5$)							
	000	001	010	011	100	101	110	111
0000	NUL	DEL	SP	0	@	P	,	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	,	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL



武汉大学
Wuhan University

谢谢!

2025-11-20

